

Uniwersytet Gdański

Inteligencja Obliczeniowa — rok akademicki 2025/2026

LLM-as-a-Judge:

Czy modele językowe nadają się
na stanowisko rekrutera technicznego?

Raport badawczy

Autor: Jakub Jurkian

Numer indeksu: 300806

Spis treści

1	Wstęp	2
1.1	Cel i pytania badawcze	2
2	Przegląd literatury	2
2.1	LLM-as-a-Judge	2
2.2	Retrieval-Augmented Generation	3
2.3	Ewaluacja systemów RAG	3
3	Architektura systemu	3
3.1	Stos technologiczny	3
3.2	Przepływ danych	4
3.3	Przełączanie modeli	5
4	Metodologia	5
4.1	Zestaw testowy	5
4.2	Badane modele	5
4.3	Tryby ewaluacji	6
4.4	Metryki	6
5	Wyniki	6
5.1	Accuracy — zestawienie główne	6
5.2	Baseline non-LLM: TF-IDF + cosine similarity	7
5.3	Accuracy per etykieta	9
5.4	Przedziały ufności (Bootstrap)	9
5.5	Stabilność wyników	11
5.6	Latencja i koszt eksperymentu	11
5.7	Gullibility — odporność na odpowiedzi adversarialne	12
6	Analiza i dyskusja	12
6.1	RAG vs. Pure Prompt	12
6.2	Porównanie modeli	13
6.3	Analiza biasu leniency	14
6.4	Niestabilność Gemini	14
7	Ograniczenia badania	15
8	Wnioski	16

1. Wstęp

Rekrutacja techniczna jest jednym z najbardziej czasochłonnych etapów w procesie zatrudnienia programistów. Ocena kandydatów wymaga nie tylko czasu rekrutera, ale przede wszystkim wiedzy eksperckiej pozwalającej zweryfikować poprawność udzielanych odpowiedzi. Pojawia się więc naturalne pytanie: czy duże modele językowe (ang. *Large Language Models*, LLM) mogłyby wspomóc lub zastąpić człowieka w tej roli?

Niniejszy raport bada paradygmat znany w literaturze jako *LLM-as-a-Judge* [1] — zastosowanie modelu językowego jako automatycznego sędziego oceniającego jakość odpowiedzi innego systemu lub człowieka. W ramach projektu zbudowano system oceniający odpowiedzi kandydatów na stanowisko Junior Java Developer, a następnie przeprowadzono serię eksperymentów porównujących trzy modele językowe z dwóch różnych rodzin: lokalny model open-source oraz modele komercyjne udostępniane przez API.

1.1. Cel i pytania badawcze

Głównym celem projektu jest ocena przydatności modeli językowych do automatycznej ewaluacji odpowiedzi technicznych. W szczególności postawiono następujące pytania badawcze:

1. Czy modele językowe potrafią rzetelnie oceniać odpowiedzi z zakresu programowania w Javie?
2. Czy zastosowanie techniki RAG (*Retrieval-Augmented Generation*) poprawia jakość oceny w porównaniu z podejściem *Pure Prompt*?
3. Czy istnieją istotne różnice w jakości oceny między modelami z różnych rodzin (lokalny vs. komercyjny)?

2. Przegląd literatury

2.1. LLM-as-a-Judge

Paradygmat *LLM-as-a-Judge* został szeroko przebadany przez Zheng et al. [1], którzy zaproponowali platformę MT-Bench do oceny modeli językowych przy użyciu silniejszego modelu jako sędziego. Autorzy wykazali, że modele GPT-4 jako sędziowie osiągają wysoką zgodność z ocenami ekspertów ludzkich, jednak zidentyfikowali również szereg tendencji (ang. *biases*), takich jak preferowanie dłuższych odpowiedzi czy błędy pozycyjne. Liu et al. [2] rozwinęli ten kierunek proponując G-Eval — metodę oceny przy użyciu GPT-4 z dodatkowym łańcuchem rozumowania (*chain-of-thought*), osiągając jeszcze wyższą zgodność z ocenami ludzkimi w zadaniach generacji tekstu. Z

kolei Wang et al. [3] pokazują, że LLM-y jako sędziowie nie są neutralne — wykazują istotny *position bias* oraz preferują odpowiedzi o określonej długości lub stylu. Aktualny przegląd literatury w tym obszarze przedstawiają Li et al. [4], systematyzując otwarte problemy metodyczne LLM-as-a-Judge. Wyniki niniejszego projektu potwierdzają inny zidentyfikowany bias — tendencję małych modeli do nadmiernej łagodności w ocenianiu (*leniency bias*).

2.2. Retrieval-Augmented Generation

Technika RAG, zaproponowana przez Lewis et al. [5], polega na wzbogacaniu promptu modelu o dokumenty pobrane z zewnętrznej bazy wiedzy na podstawie podobieństwa semantycznego. Standardowe założenie jest takie, że dodatkowy kontekst poprawia jakość odpowiedzi modelu. Aktualny przegląd ekosystemu RAG (architektury naive/advanced/modular, strategie chunkowania, re-ranking) podają Gao et al. [6]. Kluczowym komponentem retrievera są gęste reprezentacje wektorowe pytań i dokumentów; Karpukhin et al. [7] wykazali w *Dense Passage Retrieval*, że dwuwieżowy encoder trenowany kontrastywnie istotnie przewyższa BM25 w zadaniach QA. Na poziomie infrastruktury, warstwa wyszukiwania w niniejszym projekcie (pgvector) opiera się na algorytmie HNSW [8], który zapewnia logarytmiczną złożoność wyszukiwania przybliżonego najbliższego sąsiada.

W kontekście ewaluacji jakości odpowiedzi, RAG może dostarczyć modelowi wzorcową odpowiedź, względem której ocenia kandydata. Niemniej, jak wykazano w niniejszym badaniu, dla domen dobrze pokrytych przez pre-training modeli (takich jak podstawy języka Java), dodatkowy kontekst może wprowadzać zakłócenia zamiast poprawiać jakość oceny.

2.3. Ewaluacja systemów RAG

Framework RAGAS [9] dostarcza metodologię automatycznej ewaluacji systemów RAG, oceniając między innymi wierność generowanych odpowiedzi i trafność pobranych dokumentów. Niniejszy projekt przyjmuje odmienną perspektywę — zamiast ewaluować system RAG pod kątem generowania odpowiedzi, bada wpływ RAG na jakość oceny dokonywanej przez model-sędziego.

3. Architektura systemu

3.1. Stos technologiczny

System MockBean został zbudowany w oparciu o następujące technologie:

- **Java 21 + Spring Boot 3.5** — warstwa backendowa z REST API

- **LangChain4j 0.29.1** — integracja z modelami językowymi
- **PostgreSQL + pgvector** — baza wektorowa do przechowywania embeddingów
- **Ollama** — lokalne uruchamianie modelu open-source
- **Springdoc OpenAPI / Swagger UI** — interaktywna dokumentacja i ręczne testowanie endpointu
- **Python** — skrypty ewaluacyjne i analityczne (`run_evaluation.py`, `bootstrap_analysis.py`, `baseline_cosine.py`)

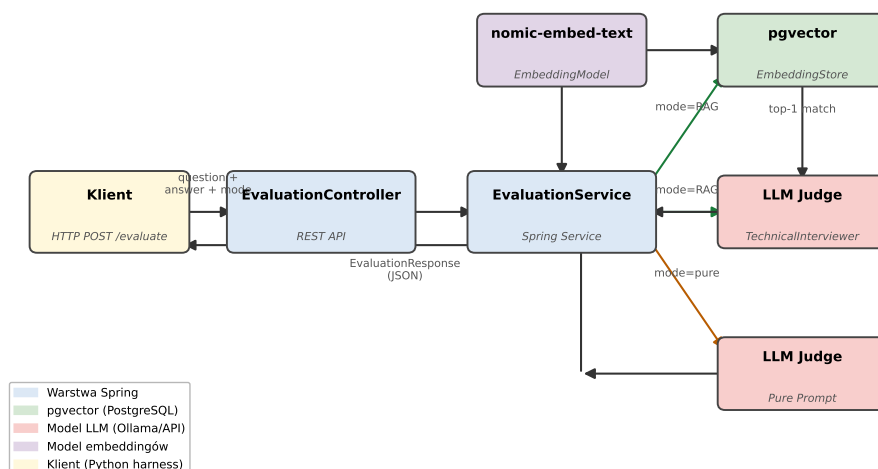
3.2. Przepływ danych

Rysunek 1 przedstawia architekturę systemu. Dla trybu RAG przepływ wygląda następująco:

1. Pytanie rekrutacyjne jest konwertowane na wektor przez model embeddingowy (`nomic-embed-text`)
2. Pgvector wyszukuje najbardziej podobną wzorcową odpowiedź (top-1)
3. Model-sędzia otrzymuje pytanie, wzorcową odpowiedź i odpowiedź kandydata
4. Model zwraca ocenę w formacie JSON: wynik liczbowy, mocne strony, pominięte koncepcje i feedback tekstowy

W trybie *Pure Prompt* krok 1 i 2 są pomijane — model ocenia wyłącznie na podstawie własnej wiedzy z pre-trainingu.

Rysunek: Architektura systemu MockBean



Rysunek 1: Architektura systemu MockBean

3.3. Przełączanie modeli

Dzięki mechanizmowi profili Spring Boot możliwe jest uruchomienie aplikacji z dowolnym z trzech modeli bez modyfikacji kodu. Każdy profil ładuje odrębny plik konfiguracyjny YAML definiujący endpoint i klucz API odpowiedniego dostawcy. Klasa konfiguracyjna oznaczona adnotacją `@Primary` zapewnia, że Spring wstrzykuje właściwy bean `ChatLanguageModel` do warstwy serwisowej.

4. Metodologia

4.1. Zestaw testowy

Na potrzeby badania stworzono ręcznie zestaw 40 par pytanie–odpowiedź z zakresu Core Java i programowania obiektowego (OOP). Każda para została opatrzona etykietą ludzką należącą do jednej z pięciu klas, po 8 przypadków na klasę (tabela 1).

Tabela 1: Klasy etykiet w zestawie testowym

Etykieta	Opis	Liczba
<code>correct</code>	Odpowiedź poprawna i wyczerpująca	8
<code>partial</code>	Odpowiedź częściowo poprawna, brakuje głębi	8
<code>wrong</code>	Odpowiedź zawiera błędy merytoryczne	8
<code>waffle</code>	Odpowiedź brzmi mądrze, ale jest merytorycznie pusta	8
<code>evasive</code>	Odpowiedź unikająca, kandydat przyznaje się do niewiedzy	8
Łącznie		40

Klasy `wrong`, `waffle` i `evasive` tworzą grupę *adwersarialną* — odpowiedzi które nie zasługują na pozytywną ocenę, ale mogą zmylić model-sędziego.

4.2. Badane modele

W eksperymencie porównano trzy modele językowe z różnych rodzin (tabela 2).

Tabela 2: Badane modele językowe

Model	Dostawca	Typ	Parametry
llama3.1:8b	Meta (Ollama)	Lokalny, open-source	8B
claude-sonnet-4-6	Anthropic	Komercyjny, API	n/d
gemini-2.5-flash	Google	Komercyjny, API	n/d

4.3. Tryby ewaluacji

Każdy model przetestowano w dwóch trybach:

- **RAG** — model otrzymuje wzorcową odpowiedź pobraną z bazy wektorowej pgvector (top-1 similarity search na embeddingach `nomic-embed-text`)
- **Pure Prompt** — model ocenia wyłącznie na podstawie własnej wiedzy, bez dostarczonego kontekstu

4.4. Metryki

Mapowanie ocen na klasy. Model-sędzia zwraca wynik liczbowy $s \in [0, 1]$. Zastosowano następujące progi klasyfikacyjne:

$$\hat{y} = \begin{cases} \text{correct} & \text{jeśli } s \geq 0,65 \\ \text{partial} & \text{jeśli } 0,30 \leq s < 0,65 \\ \text{low} & \text{jeśli } s < 0,30 \end{cases} \quad (1)$$

Etykiety ludzkie `wrong`, `waffle` i `evasive` są mapowane na klasę `low`.

Accuracy. Główna metryka: odsetek przypadków gdzie przewidziana klasa zgadza się z etykietą ludzką.

Gullibility. Odsetek odpowiedzi `waffle` i `evasive` którym model przyznał wynik $s \geq 0,5$ — miara podatności na manipulację. Pełne wyniki dla wszystkich konfiguracji podaje sekcja 5.7.

Stabilność. Każdy eksperyment powtórzono dwukrotnie przy `temperature=0.0`. Różnica accuracy między runami $|\Delta| \leq 2,5\%$ uznawana jest za stabilną.

5. Wyniki

5.1. Accuracy — zestawienie główne

Tabela 3 przedstawia wyniki accuracy dla wszystkich kombinacji modelu i trybu ewaluacji.

Tabela 3: Accuracy (%) — wszystkie modele i tryby

Model	RAG		Pure Prompt	
	Run 1	Run 2	Run 1	Run 2
llama3.1:8b	50,0%	50,0%	57,5%	57,5%
gemini-2.5-flash	80,0%	80,0%	87,5%	82,5%
claude-sonnet-4-6	82,5%	82,5%	92,5%	92,5%

5.2. Baseline non-LLM: TF-IDF + cosine similarity

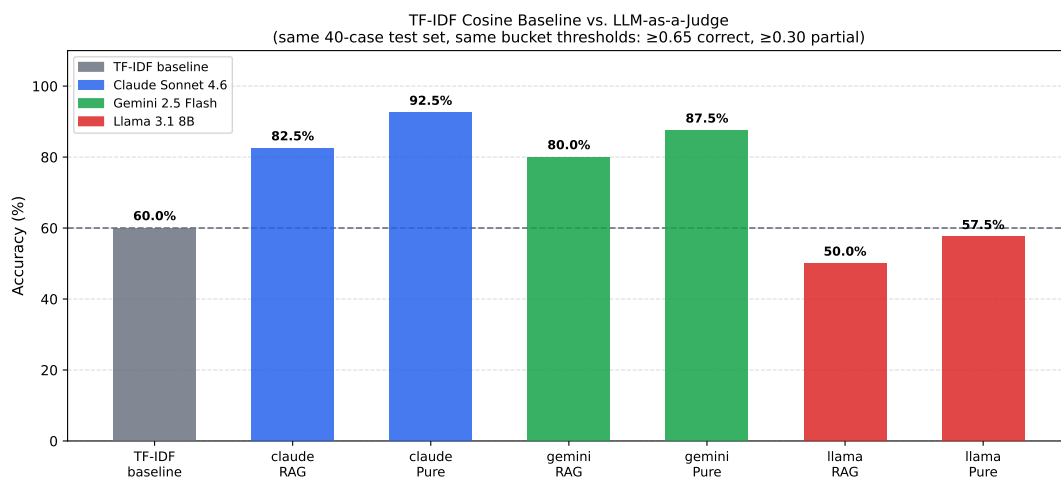
Aby umieścić wyniki modeli językowych na tle klasycznej techniki analizy tekstu, zaimplementowano w czystym Pythonie baseline oparty na klasycznej reprezentacji *bag of words* z wagami TF-IDF oraz miarą cosine similarity. Pipeline replikuje strukturę trybu RAG modeli LLM, zapewniając porównywalność:

1. Retrieval: dla pytania testowego wybierany jest najbardziej podobny wzorzec z bazy wiedzy (TF-IDF cosine między pytaniami).
2. Scoring: liczone jest cosine similarity między wektorem TF-IDF odpowiedzi kandydata a wektorem TF-IDF wzorcowej odpowiedzi z pobranego rekordu.
3. Mapowanie: wynik traktowany jest jak **score** modelu-sędziego i mapowany na bucket tymi samymi progami (0,65 / 0,30).

Baseline jest deterministyczny, darmowy (latencja < 10 ms per zapytanie) i całkowicie pozbawiony rozumienia semantycznego — sygnałem jest wyłącznie leksykalna zbieżność słów. Wyniki zestawia tabela 4 i wizualizuje rysunek 2.

Tabela 4: TF-IDF + cosine baseline vs. modele LLM w trybie Pure (accuracy per etykieta)

System	correct	partial	wrong	waffle	evasive	Overall
TF-IDF baseline	62%	100%	0%	75%	62%	60,0%
llama · Pure	100%	62%	25%	62%	38%	57,5%
gemini · Pure	88%	88%	88%	88%	88%	85,0%
claude · Pure	88%	100%	88%	88%	100%	92,5%



Rysunek 2: Porównanie accuracy: baseline TF-IDF cosinus vs. wszystkie eksperymenty LLM-as-a-Judge. Przerywana linia oznacza poziom baseline (60,0%).

Baseline osiągnął 60,0% overall accuracy, co minimalnie przewyższa `llama · Pure` (57,5%) mimo że nie używa żadnego modelu deep learning ani semantycznego rozumienia. Analiza per etykieta ujawnia ciekawą strukturę podobieństwa leksykalnego względem semantycznego:

- Na klasie **partial** baseline osiąga 100% trafności — odpowiedzi częściowo poprawne są leksykalnie najbardziej zbliżone do wzorca (kandydat używa tych samych terminów, ale brakuje mu głębi), więc trafiają dokładnie w przedział $0,30 \leq s < 0,65$ odpowiadający bucketowi **partial**.
- Na klasie **wrong** baseline kompletnie zawodzi (0%) — błędne odpowiedzi techniczne często używają tych samych terminów co poprawne (np. *garbage collector*, *synchronized*, *stream*), więc dostają wysokie podobieństwo TF-IDF i są klasyfikowane jako **correct** lub **partial**, mimo że są merytorycznie błędne. LLM rozróżnia znaczenia; TF-IDF nie.
- Na klasach **waffle** (75%) i **evasive** (62%) baseline radzi sobie umiarkowanie dobrze — odpowiedzi puste lub uchylne mają mniejsze pokrycie słownikowe z wzorcem i częściej spadają w bucket **low**, ale nie zawsze (część odpowiedzi **waffle** przemyca dość terminów technicznych, by przekroczyć próg 0,30).
- Na klasie **correct** (62%) baseline trafia tylko część poprawnych odpowiedzi — kandydaci używają synonimów i innej struktury zdań niż wzorzec, co obniża podobieństwo leksykalne poniżej progu 0,65.

Najlepszy model LLM (`claude-sonnet-4-6`, tryb Pure Prompt) uzyskuje 92,5%, co daje **przewagę** +32,5 **pp** nad baseline’em i potwierdza, że modele językowe wnoszą

realną wartość dodaną — potrafią oceniać merytoryczną poprawność odpowiedzi, a nie tylko powierzchowne podobieństwo słowne. Wyjątkiem pozostaje `llama · RAG`, która przy *accuracy* 50% nieznacznie wyprzedza *random baseline* ($\frac{1}{3} \approx 33,3\%$ przy trójklasowej klasyfikacji) i wyraźnie ustępuje TF-IDF.

Wniosek pragmatyczny. W systemie produkcyjnym sensowna byłaby hybryda: szybki filtr TF-IDF (latencja < 10 ms, koszt zerowy) pozwala odrzucić znaczną część oczywiście pustych odpowiedzi, a kosztowne wywołania LLM rezerwowane są dla przypadków wątpliwych. Taki układ pozwala obniżyć koszty API bez istotnej utraty trafności klasyfikacji.

5.3. Accuracy per etykieta

Tabela 5 przedstawia dokładność klasyfikacji z podziałem na poszczególne klasy (wartości uśrednione z dwóch runów; dla `gemini · Pure`, gdzie runy różnią się o 5 pp, wartości w kolumnie **Overall** są średnią arytmetyczną Run 1 i Run 2, co daje 85,0%).

Tabela 5: Accuracy per etykieta (%)

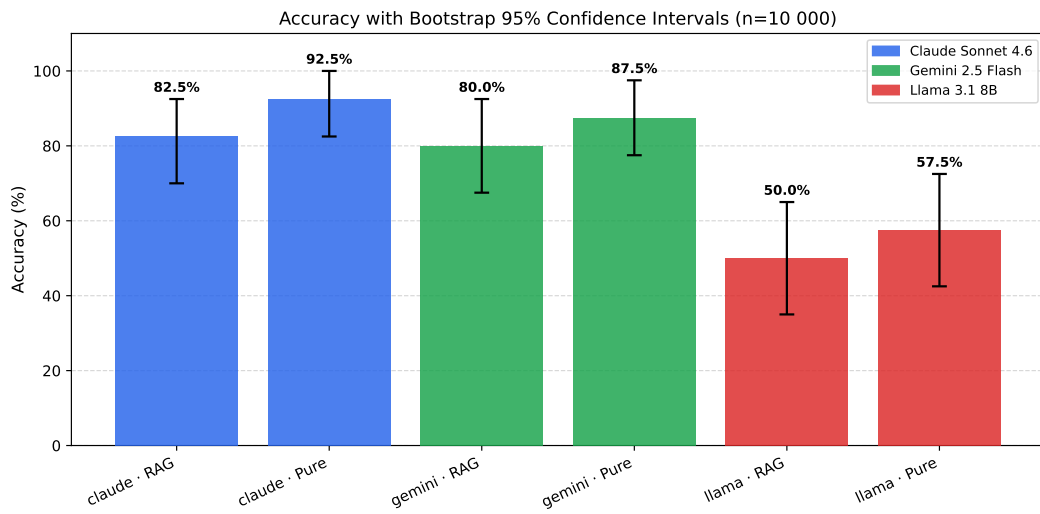
Model / Tryb	correct	partial	wrong	waffle	evasive	Overall
llama · RAG	100%	25%	12%	50%	62%	50,0%
llama · Pure	100%	62%	25%	62%	38%	57,5%
gemini · RAG	88%	62%	88%	88%	75%	80,0%
gemini · Pure	82%	82%	88%	88%	88%	85,0%
claude · RAG	88%	62%	88%	88%	88%	82,5%
claude · Pure	88%	100%	88%	88%	100%	92,5%

5.4. Przedziały ufności (Bootstrap)

Ze względu na ograniczony rozmiar zestawu testowego ($n = 40$) obliczono bootstrapowe 95% przedziały ufności dla *accuracy* każdego eksperymentu ($B = 10\,000$ prób, ziarno 42, próbkowanie ze zwracaniem). Przedziały ufności w tabeli 6 liczone są na danych z Run 1 każdego eksperymentu; dla pięciu z sześciu konfiguracji Run 2 daje identyczny zestaw ocen (patrz tabela 7), więc wynik bootstrapowy byłby nieodróżnialny. Wyjątkiem jest `gemini · Pure`, gdzie $\Delta = 5$ pp między runami — wartości w tabeli odnoszą się tu konkretnie do Run 1 (87,5%).

Tabela 6: Accuracy z bootstrapowym 95% przedziałem ufności ($n = 10\,000$ prób)

Eksperyment	Accuracy	CI dolny	CI górny	$\pm$ half
claude · RAG	82,5%	70,0%	92,5%	$\pm 11,2\%$
claude · Pure	92,5%	82,5%	100,0%	$\pm 8,8\%$
gemini · RAG	80,0%	67,5%	92,5%	$\pm 12,5\%$
gemini · Pure	87,5%	77,5%	97,5%	$\pm 10,0\%$
llama · RAG	50,0%	35,0%	65,0%	$\pm 15,0\%$
llama · Pure	57,5%	42,5%	72,5%	$\pm 15,0\%$



Rysunek 3: Accuracy z bootstrapowymi 95% przedziałami ufności dla wszystkich eksperymentów

Szerokość przedziałów ($\pm 8,8$ – $15,0$ pp) odzwierciedla ograniczony rozmiar zestawu testowego i należy ją uwzględnić przy interpretacji wyników. Mimo to claude · Pure konsekwentnie wyprzedza pozostałe eksperymenty, a dolna granica jego CI (82,5%) jest wyższa niż górna granica llama · Pure (72,5%), co wskazuje na statystycznie istotną różnicę między tymi dwoma modelami.

5.5. Stabilność wyników

Tabela 7: Stabilność wyników — różnica między runami

Eksperyment	Run 1	Run 2	Δ	Ocena
llama · RAG	50,0%	50,0%	0,0%	Stabilny
llama · Pure	57,5%	57,5%	0,0%	Stabilny
gemini · RAG	80,0%	80,0%	0,0%	Stabilny
gemini · Pure	87,5%	82,5%	5,0%	Niestabilny
claude · RAG	82,5%	82,5%	0,0%	Stabilny
claude · Pure	92,5%	92,5%	0,0%	Stabilny

5.6. Latencja i koszt eksperymentu

Z punktu widzenia wdrożenia produkcyjnego sama trafność nie wystarcza — istotne są również czas odpowiedzi (od momentu wysłania żądania do otrzymania oceny w JSON) oraz koszt jednej oceny. Tabela 8 podsumowuje statystyki latencji obliczone na danych z Run 1 ($n = 40$ wywołań).

Tabela 8: Latencja end-to-end pojedynczej oceny (ms) — Run 1, $n = 40$

Eksperyment	średnia	mediana	p95	maks.	Rozliczenie
claude · RAG	6 177	6 138	8 045	8 711	API (Anthropic)
claude · Pure	7 496	7 474	9 106	9 558	API (Anthropic)
gemini · RAG	6 256	5 958	10 119	11 683	API (Google)
gemini · Pure	6 765	6 804	8 364	9 183	API (Google)
llama · RAG	23 396	22 162	35 222	37 163	lokalnie (CPU)
llama · Pure	25 649	25 204	35 913	45 176	lokalnie (CPU)

Łączny czas eksperymentu. Pełne odtworzenie pojedynczego runu (40 wywołań sekwencyjnych) trwa: claude $\approx$ 4–5 min, gemini $\approx$ 4–4,5 min, llama $\approx$ 15,5–17 min. Llama jest tu istotnie wolniejsza, ponieważ działa lokalnie przez Ollama na CPU — z dostępem do GPU różnica zmniejszyłaby się do akceptowalnego poziomu.

Koszt. Modele API rozliczane są od liczby tokenów wejściowych i wyjściowych. Średnia długość promptu w trybie RAG to ok. 500–700 tokenów (system message + pytanie + wzorzec + odpowiedź kandydata), a odpowiedź JSON ok. 300–500 tokenów. Przyjmując publiczne stawki obowiązujące w momencie eksperymentów (Claude Sonnet 4.6: 3 USD/1M input, 15 USD/1M output; Gemini 2.5 Flash: 0,30/2,50 USD analogicznie), koszt 40 ocen mieści się w okolicy: $\sim$ 0,30 USD dla Claude i $\sim$ 0,03 USD dla Gemini.

Llama3.1:8b uruchamiana przez Ollama lokalnie jest darmowa w sensie pieniężnym, ale generuje wyraźny koszt czasowy.

Trade-off. Zestawienie z tabelą 3 pokazuje wyraźny pareto-front: Claude osiąga najwyższą accuracy przy najwyższym koszcie pieniężnym, Gemini daje niemal porównywalną trafność za jedną dziesiątą ceny, a Llama jest opcją „darmową, ale niedokładną i wolną”. Dla scenariusza ewaluacyjnego, w którym jakość oceny przekłada się bezpośrednio na decyzję rekrutacyjną, koszt API jest pomijalny i wybór Claude jest uzasadniony.

5.7. Gullibility — odporność na odpowiedzi adversarialne

Metryka *gullibility*, zdefiniowana w sekcji 4.4, mierzy odsetek odpowiedzi z klas **waffle** (puste, ale brzmiące mądrze) oraz **evasive** (uchylne, „nie wiem”), którym sędzia przyznał wynik $s \geq 0,5$. Niska wartość świadczy o wysokiej odporności sędziego na próby zmylenia. Zestawienie dla wszystkich siedmiu konfiguracji (6 LLM-owych + baseline TF-IDF) podaje tabela 9.

Tabela 9: Gullibility (% odpowiedzi adversarialnych z $s \geq 0,5$, Run 1, $n_{adv} = 16$)

System	waffle ($\geq 0,5$)	evasive ($\geq 0,5$)	gullibility	Komentarz
TF-IDF baseline	0/8	0/8	0,0%	ślepy na semantykę, odporny z
claude · RAG	0/8	0/8	0,0%	idealna selektywność
claude · Pure	0/8	0/8	0,0%	idealna selektywność
gemini · Pure	0/8	0/8	0,0%	odporny w trybie Pure
llama · RAG	0/8	0/8	0,0%	artefakt rozkładu — patrz dysk
gemini · RAG	0/8	1/8	6,2%	jedna evasive zmyliła sędziego
llama · Pure	1/8	1/8	12,5%	najsłabszy filtr semantyczny

Modele komercyjne wykazują pełną odporność na adversarialne odpowiedzi. Llama w trybie Pure przepuszcza 12,5% takich przypadków — to spójne z obserwowanym dla niej *leniency bias* (sekcja 6.3). Zerowa gullibility Llamy w trybie RAG jest artefaktem rozkładu ocen, a nie rzeczywistą czujnością: model po prostu rzadko przyznaje wynik powyżej 0,5 niezależnie od jakości odpowiedzi.

6. Analiza i dyskusja

6.1. RAG vs. Pure Prompt

Najważniejszym i najbardziej zaskakującym wynikiem badania jest fakt, że tryb *Pure Prompt* przewyższa tryb RAG dla każdego z trzech badanych modeli (tabela 3). Wzrost accuracy wynosi od +7,5 pp dla modeli llama i gemini, do +10 pp dla Claude.

Hipoteza wyjaśniająca to zjawisko jest następująca: baza wiedzy dostarcza wzorcową odpowiedź pobraną metodą top-1 similarity search. Jeśli pobrany dokument nie jest semantycznie idealnie dopasowany do pytania, model-sędzia otrzymuje mylący punkt odniesienia i może nagradzać odpowiedzi za leksykalne podobieństwo do wzorca zamiast oceniać merytoryczną poprawność. Nazywamy to zjawisko *biasem partial match*.

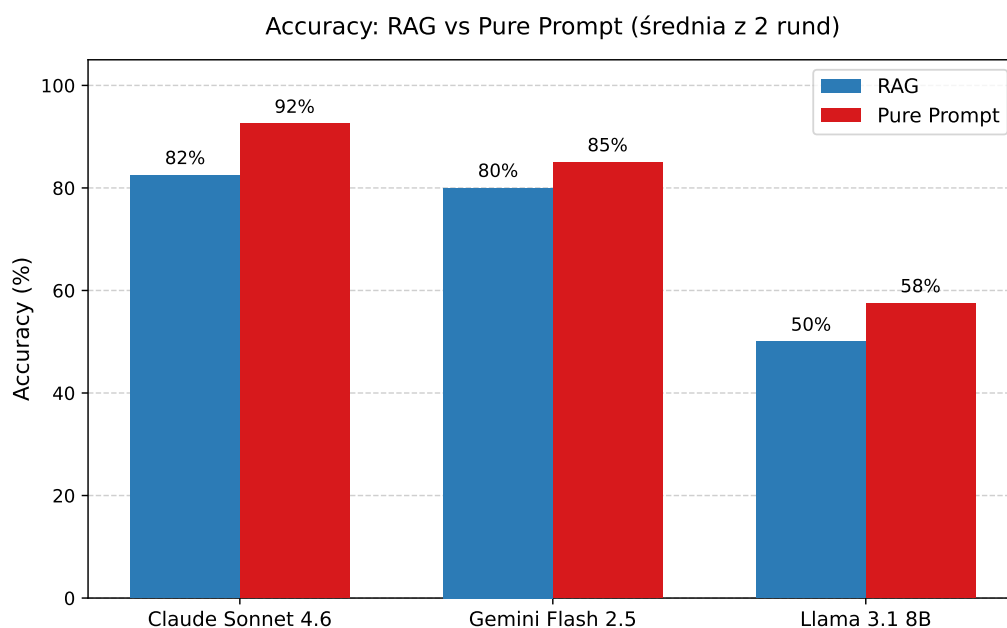
W trybie *Pure Prompt* model ocenia na podstawie własnej wiedzy z pre-trainingu — dla zagadnień z zakresu podstaw języka Java, które są wyjątkowo dobrze pokryte w danych treningowych dużych modeli, okazuje się to skuteczniejszą strategią.

Wynik ten jest jednak specyficzny dla badanej domeny i nie powinien być uogólniany na domeny niszowe lub wymagające aktualnej wiedzy, gdzie RAG jest niezastąpiony.

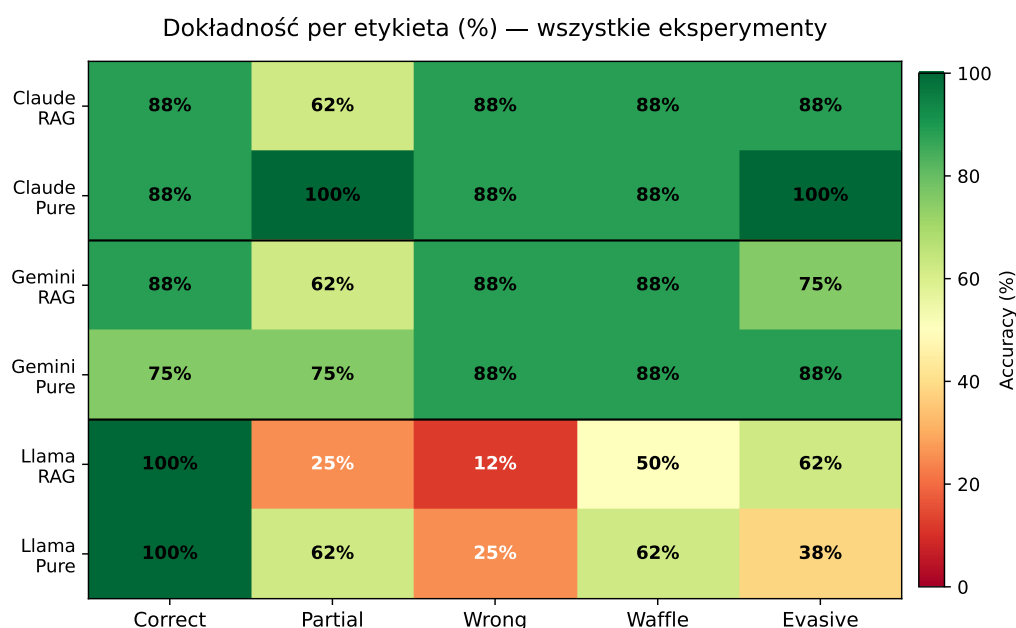
6.2. Porównanie modeli

Wyniki wskazują na wyraźne różnice między modelami (rysunek 4):

- **claude-sonnet-4-6** osiągnął najwyższą accuracy (92,5% w trybie Pure), wykazując przy tym pełną stabilność ($\Delta = 0\%$)
- **gemini-2.5-flash** uzyskał zbliżone wyniki (85% średnio w trybie Pure), jednak z pewną niestabilnością ($\Delta = 5\%$ między runami)
- **llama3.1:8b** wyraźnie odstaje od modeli komercyjnych nawet w najlepszym trybie (57,5%), co wskazuje że rozmiar i jakość modelu mają kluczowe znaczenie dla zadania LLM-as-a-Judge



Rysunek 4: Accuracy modeli w trybie RAG i Pure Prompt (średnia z 2 rund)



Rysunek 5: Dokładność per etykieta (%) dla wszystkich kombinacji model × tryb

Rysunek 5 przedstawia szczegółowy rozkład dokładności dla każdej etykiety i każdego eksperymentu. Widoczna jest wyraźna asymetria: klasy **partial** oraz **wrong** sprawiają modelom największe trudności, podczas gdy klasa **correct** jest rozpoznawana niemal bezbłędnie.

6.3. Analiza biasu leniency

Llama3.1:8b w trybie RAG wykazuje szczególnie niską accuracy dla klasy **wrong** (12%) — model prawie nigdy nie wykrywa merytorycznie błędnych odpowiedzi. Zjawisko to, znane w literaturze jako *leniency bias* [1], polega na systematycznym zawyżaniu ocen przez model-sędziego.

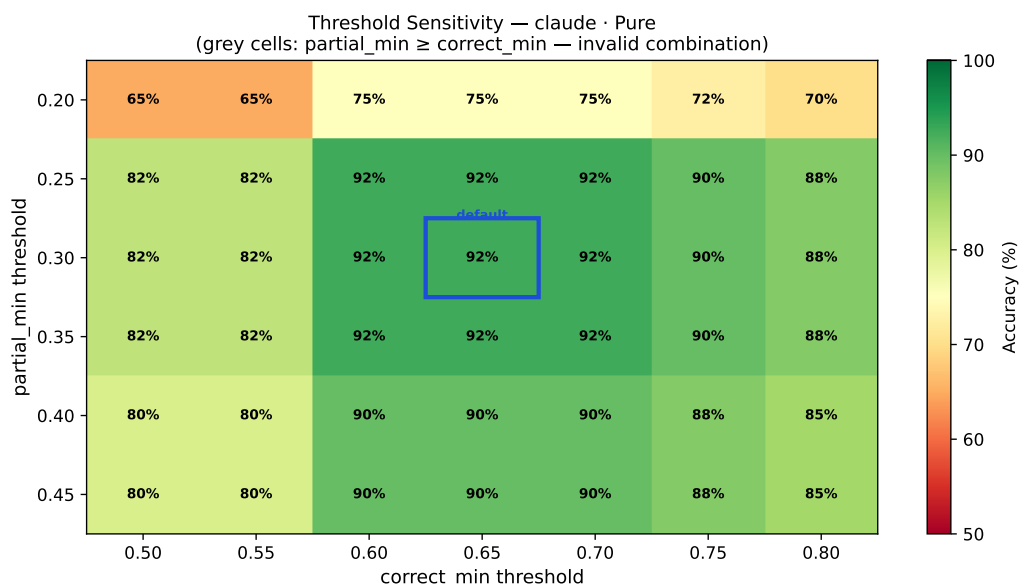
Paradoksalnie, llama w trybie RAG osiąga 0% gullibility (brak odpowiedzi **waffle**/**evasive** z wynikiem $\geq 0,5$) — nie przez skuteczne wykrywanie manipulacji, lecz dlatego że żadna odpowiedź nie przekroczyła progu 0,5, co jest artefaktem rozkładu ocen, a nie rzeczywistą czujnością modelu.

6.4. Niestabilność Gemini

Przy ustawieniu `temperature=0.0`, llama i Claude wykazują pełną deterministyczność ($\Delta = 0\%$). Gemini-2.5-flash w trybie Pure Prompt wykazuje różnicę 5 pp między runami, co wskazuje że Google nie gwarantuje pełnej powtarzalności wyników nawet przy zerowej temperaturze — prawdopodobnie z powodu niedeterministycznej infrastruktury serwerowej po stronie API. Jest to istotna uwaga praktyczna przy wdrożeniach produkcyjnych systemów opartych na tym modelu.

7. Ograniczenia badania

1. **Domena.** Badanie ogranicza się do zagadnień Core Java i OOP — dziedzin doskonale pokrytych przez pre-training dużych modeli. W domenach niszowych lub wymagających aktualnej wiedzy wyniki mogłyby być odmienne, a RAG mógłby okazać się korzystny.
2. **Strategia retrieval.** Zastosowano top-1 similarity search. Podejście top- k (np. $k = 3$) mogłoby poprawić jakość RAG i zmienić wnioski dotyczące porównania trybów.
3. **Rozmiar zestawu testowego.** 40 przypadków testowych pozwala wyciągać wstępne wnioski, jednak dla wyników statystycznie istotnych wymagany byłby większy zestaw danych.
4. **Brak human baseline.** Progi klasyfikacyjne i etykiety ludzkie zostały wyznaczone przez autora bez walidacji przez niezależnych ekspertów. Nie przeprowadzono bezpośredniego porównania z ocenami prawdziwego rekrutera.
5. **Kalibracja progów.** Zastosowane progi (0,65 i 0,30) są stałe dla wszystkich modeli, mimo że rozkłady ocen różnią się między modelami (np. średnia ocena llamy to 0,517, a Gemini 0,384). Przeprowadzona analiza wrażliwości (rysunek 6) pokazuje, że najlepszy wynik (92,5%) jest stabilny w szerokim zakresie progów dla Claude Pure, co potwierdza trafność wybranych wartości.



Rysunek 6: Analiza wrażliwości progów klasyfikacyjnych dla eksperymentu claude · Pure. Niebieska ramka oznacza progi zastosowane w badaniu (0,65 / 0,30). Szare komórki to kombinacje niepoprawne ($\text{partial_min} \geq \text{correct_min}$).

8. Wnioski

Niniejszy raport przedstawił wyniki badania oceniającego przydatność modeli językowych do automatycznej ewaluacji odpowiedzi technicznych w kontekście rekrutacji na stanowisko Junior Java Developer.

Najważniejsze wnioski są następujące:

1. **Modele komercyjne są skutecznymi sędziami technicznymi.** Claude-sonnet-4-6 (92,5%) i gemini-2.5-flash (85%) osiągają wysoką accuracy, co czyni je obiecującymi narzędziami wspomagającymi rekrutację.
2. **RAG jest kontrproduktywny dla dobrze znanych domen.** We wszystkich trzech badanych modelach tryb Pure Prompt przewyższa RAG, co sugeruje że dla zagadnień dobrze pokrytych przez pre-training, dodatkowy kontekst wprowadza bias partial match zamiast poprawiać ocenę.
3. **Rozmiar modelu ma kluczowe znaczenie.** Llama3.1:8b, mimo że jest modelem open-source działającym lokalnie, pozostaje znacząco słabszym sędzią (57,5%) niż modele komercyjne. Leniency bias sprawia, że model nagradza błędne odpowiedzi, co dyskwalifikuje go w roli autonomicznego sędziego rekrutacyjnego.
4. **Deterministyczność jest ważna w zastosowaniach ewaluacyjnych.** Różnica w stabilności między Claude (pełny determinizm) a Gemini ($\Delta = 5\%$) jest istotna praktycznie — powtarzalność wyników jest wymagana w rzetelnym procesie rekrutacyjnym.

Odpowiadając na tytułowe pytanie: *duże modele komercyjne* wykazują potencjał jako narzędzie wspomagające rekrutera technicznego, jednak ze względu na zidentyfikowane ograniczenia (brak human baseline, możliwy bias) powinny pełnić rolę uzupełniającą, a nie zastępować ocenę eksperta.

Kierunkiem przyszłych badań mogłoby być porównanie ocen modeli z ocenami prawdziwych rekruterów, co pozwoliłoby bezpośrednio odpowiedzieć na tytułowe pytanie.

Bibliografia

- [1] Lianmin Zheng i in. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. arXiv:2306.05685. 2023. arXiv: [2306.05685 \[cs.CL\]](https://arxiv.org/abs/2306.05685). URL: <https://arxiv.org/abs/2306.05685>.
- [2] Yang Liu i in. *G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment*. arXiv:2303.16634. 2023. arXiv: [2303.16634 \[cs.CL\]](https://arxiv.org/abs/2303.16634). URL: <https://arxiv.org/abs/2303.16634>.

- [3] Peiyi Wang i in. *Large Language Models are not Fair Evaluators*. arXiv:2305.17926. 2023. arXiv: [2305.17926 \[cs.CL\]](https://arxiv.org/abs/2305.17926). URL: <https://arxiv.org/abs/2305.17926>.
- [4] Dawei Li i in. *From Generation to Judgment: Opportunities and Challenges of LLM-as-a-Judge*. arXiv:2411.16594. 2024. arXiv: [2411.16594 \[cs.CL\]](https://arxiv.org/abs/2411.16594). URL: <https://arxiv.org/abs/2411.16594>.
- [5] Patrick Lewis i in. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. W: *Advances in Neural Information Processing Systems*. T. 33. Curran Associates, Inc., 2020, s. 9459–9474. URL: <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- [6] Yunfan Gao i in. *Retrieval-Augmented Generation for Large Language Models: A Survey*. arXiv:2312.10997. 2024. arXiv: [2312.10997 \[cs.CL\]](https://arxiv.org/abs/2312.10997). URL: <https://arxiv.org/abs/2312.10997>.
- [7] Vladimir Karpukhin i in. “Dense Passage Retrieval for Open-Domain Question Answering”. W: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 2020, s. 6769–6781. URL: <https://aclanthology.org/2020.emnlp-main.550/>.
- [8] Yu A. Malkov i D. A. Yashunin. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs”. W: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2020), s. 824–836. DOI: [10.1109/TPAMI.2018.2889473](https://doi.org/10.1109/TPAMI.2018.2889473).
- [9] Shahul Es i in. *RAGAS: Automated Evaluation of Retrieval Augmented Generation*. arXiv:2309.15217. 2023. arXiv: [2309.15217 \[cs.CL\]](https://arxiv.org/abs/2309.15217). URL: <https://arxiv.org/abs/2309.15217>.